

## Estruturas, Dados e Operação das Funcionalidades

### Objetivo das Estruturas e Dados

Esta seção do projeto detalha os dados empregados pelo Gerador de Senhas Aleatórias, sua organização de armazenamento e o modo como cada funcionalidade do menu os utiliza. As definições seguem os objetivos e requisitos do projeto, bem como os fluxogramas estabelecidos.

As estruturas são implementadas no arquivo senha.h, as funções que as manipulam estão em senha.c, e o menu principal reside em main.c.

### Constantes do Sistema

Os limites operacionais do sistema são definidos por meio de #define em senha.h. Esta abordagem permite centralizar modificações futuras em um único ponto.

- TAM\_MIN\_SENHA (1): Indica o menor tamanho de senha aceito.
- TAM\_MAX\_SENHA (32): Indica o maior tamanho de senha aceito.
- MAX\_SENHAS (100): Define a quantidade máxima de senhas armazenáveis em memória.
- ARQUIVO\_SENHAS ("senhas.txt"): Nome do arquivo para registro persistente das senhas.
- ARQUIVO\_TEMP ("senhas\_temp.txt"): Arquivo auxiliar empregado na exclusão de senhas.

### Estrutura Senha

Esta estrutura armazena os dados de uma senha gerada pelo sistema.

```
struct Senha {  
    int id;  
    char texto[TAM_MAX_SENHA + 1];  
    int tamanho;  
};
```

### Campos

- id (int): Identificador sequencial da senha (por exemplo: 1, 2, 3).
- texto (char[33]): A senha gerada, composta por 32 posições para caracteres e uma para o terminador nulo (\0).
- tamanho (int): O número de caracteres da senha, variando de 1 a 32.

### Estrutura ConfiguracaoSenha

Esta estrutura captura as escolhas do usuário na opção "Criar senha". Para as categorias, o valor 1 indica a seleção do tipo de caractere, enquanto 0 indica sua exclusão.

```

struct ConfiguracaoSenha {
    int tamanho;
    int usarMaiusculas;
    int usarMinusculas;
    int usarNumeros;
    int usarEspeciais;
};

```

### **Campos**

- tamanho (int): O tamanho desejado para a senha.
- usarMaiusculas (int): Indica a inclusão de letras maiúsculas.
- usarMinusculas (int): Indica a inclusão de letras minúsculas.
- usarNumeros (int): Indica a inclusão de números.
- usarEspeciais (int): Indica a inclusão de caracteres especiais.

Caso todos os quatro campos de categoria sejam 0, nenhuma categoria foi selecionada. O sistema então solicita novamente as escolhas, conforme ilustrado no fluxograma de criação de senha.

## **Estrutura ListaSenhas**

Esta estrutura gerencia as senhas residentes em memória durante a execução do programa. Inclui um vetor de estruturas do tipo Senha, o número de senhas armazenadas e o próximo ID a ser atribuído.

```

struct ListaSenhas {
    struct Senha senhas[MAX_SENHAS];
    int quantidade;
    int proximoId;
};

```

### **Campos**

- senhas (vetor de struct Senha): Contém as senhas geradas na execução atual.
- quantidade (int): O número de senhas atualmente no vetor, inicializado em 0.
- proximoId (int): O ID que será atribuído à próxima senha.

Ao iniciar o programa, verifica-se a existência do arquivo senhas.txt. Se presente, o sistema lê o arquivo, determina o maior ID registrado e incrementa-o em um para definir proximoId. Se o arquivo não existir, proximoId é inicializado em 1. Esta lógica previne a atribuição de IDs já existentes.

A lista é declarada em main.c e passada às funções por ponteiro (&lista), permitindo que as funções modifiquem seus dados.

## **Caracteres Utilizados na Senha**

Cada categoria de caractere é representada por uma string contendo os caracteres permitidos:

- **Letras maiúsculas:** ABCDEFGHIJKLMNOPQRSTUVWXYZ
- **Letras minúsculas:** abcdefghijklmnopqrstuvwxyz
- **Números:** 0123456789
- **Caracteres especiais:** !@#\$%&\*()-\_+=[]{}?

As categorias selecionadas pelo usuário são combinadas em um vetor denominado conjunto. Cada caractere da senha é então sorteado deste vetor usando `rand() % tamanhoConjunto`. A função `srand(time(NULL))` é invocada uma única vez no início do programa para assegurar a geração de senhas distintas em cada execução.

O ponto e vírgula e o espaço são excluídos dos caracteres especiais. O ponto e vírgula serve como separador de dados no arquivo, e o espaço comprometeria a legibilidade da senha.

## Arquivo Local

As senhas registradas são armazenadas em `senhas.txt`. Cada linha do arquivo contém o ID e a senha, separados por um ponto e vírgula. Exemplos incluem:

```
1;aB3$kP9!  
2;7713904  
3;Qw#e2Rt@Yu8i
```

A gravação é realizada com `fprintf`, e a leitura com `fscanf`. Após cada operação, o arquivo é fechado via `fclose`.

### Modo de Abertura em Cada Funcionalidade

- **Início do programa:** "r", para leitura dos IDs já registrados.
- **Registrar senhas:** "a", para adicionar senhas ao final do arquivo sem sobrescrever as existentes.
- **Verificar senhas:** "r", para leitura das senhas registradas.
- **Apagar senha:** "r" para `senhas.txt` e "w" para `senhas_temp.txt`.

## Operação das Funcionalidades

### Criar Senha

- O usuário fornece o tamanho e as categorias desejadas, que são armazenadas na estrutura `ConfiguracaoSenha`.
- O sistema constrói o conjunto de caracteres válidos e sorteia cada caractere para compor a senha.
- A senha gerada recebe o `proximoId` e é inserida no vetor da `ListaSenhas`.
- Os valores de quantidade e `proximoId` são incrementados em 1.
- O sistema exibe o ID e a senha gerada.

### Registrar Senhas em Arquivo

- O sistema abre `senhas.txt` e grava cada senha da memória no formato `id;senha`.
- Após a gravação, quantidade é redefinido para 0, evitando a gravação duplicada das mesmas senhas.
- O sistema informa o número de senhas registradas.

### Verificar Senhas

- O sistema lê e exibe as senhas contidas no arquivo.
- Posteriormente, exibe as senhas em memória cujos IDs ainda não apareceram no arquivo, evitando repetições na lista.

## Apagar Senha

- O usuário informa o ID da senha a ser apagada.
- O sistema busca o ID no vetor em memória. Se encontrado, as senhas subsequentes são deslocadas uma posição para trás, e quantidade é decrementado em 1.
- O sistema também busca o ID no arquivo, mesmo que já tenha sido encontrado em memória. Todas as linhas com IDs diferentes são copiadas para senhas\_temp.txt. Em seguida, senhas.txt é removido com remove, e o arquivo temporário é renomeado para senhas.txt com rename. Este procedimento é necessário, pois a linguagem C não permite a exclusão direta de uma linha intermediária em um arquivo.
- O sistema informa se a senha foi apagada ou se o ID não foi localizado.

## Sair do Programa

Esta funcionalidade não requer dados adicionais. Os arquivos são fechados ao término de cada operação, permitindo o encerramento do programa sem perda das senhas registradas.

O sistema não inclui uma opção para alterar uma senha. Se o usuário necessitar de uma senha diferente, deve criar uma nova e, opcionalmente, apagar a antiga pelo ID.

## Validações

- **Opção do menu:** Aceita números de 1 a 5.
- **Tamanho da senha:** Aceita números de 1 a 32.
- **Categorias:** Respostas 1 (sim) ou 0 (não), com a exigência de que pelo menos uma categoria seja escolhida.
- **Memória:** Senhas são criadas somente se quantidade for menor que MAX\_SENHAS.
- **Registro:** O registro de senhas ocorre apenas se houver pelo menos uma senha em memória.
- **ID para apagar:** Deve ser um número maior que 0.
- **Arquivo:** Se fopen retornar NULL, uma mensagem de erro é exibida ao usuário, que então retorna ao menu.

Quando o usuário insere um caractere não numérico onde um número é esperado, a entrada é descartada e um novo valor é solicitado, prevenindo travamentos ou fechamentos inesperados do programa.

## Relação com os Conteúdos da Disciplina

- **Entrada e saída:** Utilização de printf e scanf no menu e nas funcionalidades.
- **Estruturas de controle:** do-while para o menu, switch para as opções, if para as validações e for na geração da senha.
- **Funções:** Cada opção do menu possui sua função correspondente em senha.c.
- **Estruturas homogêneas:** Strings das categorias, o vetor conjunto e o texto da senha.
- **Estruturas heterogêneas:** struct Senha, struct ConfiguracaoSenha e o vetor de struct Senha dentro de ListaSenhas.
- **Arquivos:** Operações de abertura, leitura, escrita e fechamento de senhas.txt e senhas\_temp.txt.
- **Divisão em arquivos:** Organização em main.c, senha.h e senha.c.